Q1: First Palindromic String

Aim

To locate the earliest string that reads identically from left to right and right to left.

Algorithm

1. Read the list of words.
2. Check each word against its reverse.
3. Return the first matching word.
4. If none matches, return an empty string.

Input

words = [abc , car, ada , racecar , cool]

Output

Ada

```
n = int(input("Enter number of words: "))

words = []
for i in range(n):
    words.append(input("Enter word: "))

answer = ""

for word in words:
    if word == word[::-1]:
        answer = word
        break

print("First palindromic string:", answer)
```

```
Output

Enter number of words: 3
Enter word: ada
Enter word: book
Enter word: car
First palindromic string: ada
```

Time Complexity: O(nL)

Space Complexity: O(1)

Result

The program returns the first palindromic word, "ada".

Q2: Count Common Values in Two Arrays

Aim

To count array positions whose values also occur in the other array.

Algorithm

1. Convert each array to a set for membership checks.
2. Count nums1 elements present in set(nums2).
3. Count nums2 elements present in set(nums1).

4. Return both counts.

Input

nums1 = [2, 3, 2], nums2 = [1, 2]

Output

[2, 1]

```python
a = list(map(int, input("Enter nums1: ").split()))
b = list(map(int, input("Enter nums2: ").split()))
s1, s2 = set(a), set(b)
answer1 = sum(x in s2 for x in a)
answer2 = sum(x in s1 for x in b)
print([answer1, answer2])
```

```
Output

Enter nums1: 2 3 2
Enter nums2: 1 2
[2, 1]
```

Time Complexity: O(n+m) average

Space Complexity: O(n+m)

Result

The returned counts are [2, 1].

Q3: Sum of Squared Distinct Counts of All Subarrays

Aim

To calculate the total of squared distinct-element counts for every non-empty subarray.

Algorithm

1. Start from each index i.

2. Extend the subarray one element at a time.

3. Maintain a set of distinct values.

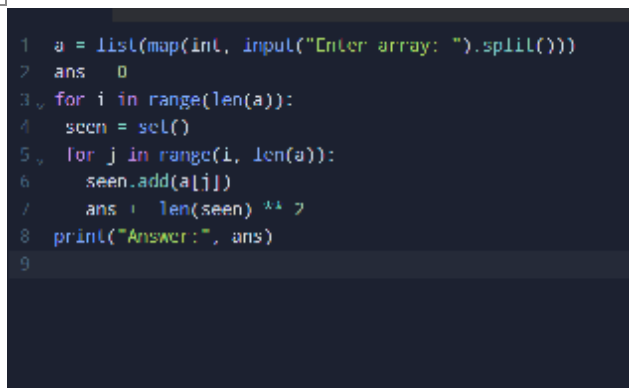
4. Add the square of the set size to the total.

Input

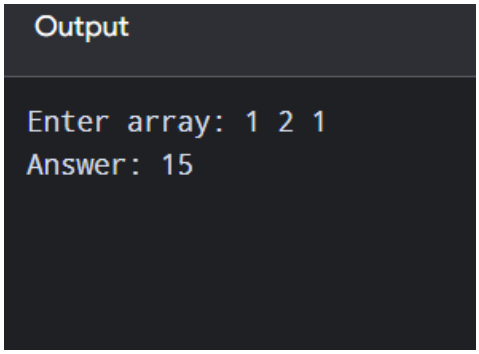
nums = [1, 2, 1]

Output

15



```
a = list(map(int, input("Enter array: ").split()))
ans = 0
for i in range(len(a)):
    seen = set()
    for j in range(i, len(a)):
        seen.add(a[j])
        ans += len(seen) ** 2
print("Answer:", ans)
```



```
Output

Enter array: 1 2 1
Answer: 15
```

Time Complexity: O(n²)

Space Complexity: O(n)

Result

The sum of squared distinct counts is
15.

Q4: Equal-Value Index Pairs Divisible by k

Aim

To count pairs of equal values whose index product is divisible by k.

Algorithm

1. Examine every pair i, j with i < j.

2. Check nums[i] == nums[j].

3. Check (i * j) % k == 0.

4. Increase the count for every valid pair.

Input

nums = [3, 1, 2, 2, 2, 1, 3], k = 2

Output

4

main.py

```
a = list(map(int, input("Enter array: ").split()))
k = int(input("Enter k: "))
ans = 0
for i in range(len(a)):
    for j in range(i + 1, len(a)):
        if a[i] == a[j] and (i * j) % k == 0:
            ans += 1
print("Number of pairs:", ans)
```

Output

```
Enter array: 3 1 2 2 2 1 3
Enter k: 2
Number of pairs: 4
```

Time Complexity: O(n²)

Space Complexity: O(1)

Result

There are 4 valid index pairs.

Q5: Find the Maximum Element

Aim

To identify the largest value in a list using one scan.

Algorithm

1. If the list is empty, report that no maximum exists.
2. Set maximum to the first element.
3. Compare each remaining element with maximum.
4. Return the largest value.

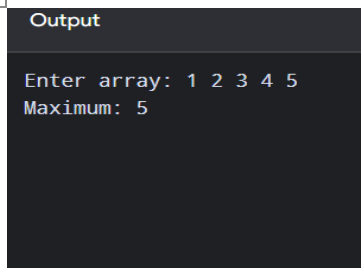
Input

1 2 3 4 5

Output:

5

```
a = list(map(int, input("Enter array: ").split()))
print("Maximum:", max(a))
```



Time Complexity: O(n)

Space Complexity: O(1)

Result

The largest element is found in a single traversal.

Q6: Sort Then Find Maximum

Aim

To sort a list with an efficient built-in sorting routine and retrieve its maximum safely.

Algorithm

1. If the list is empty, return None.

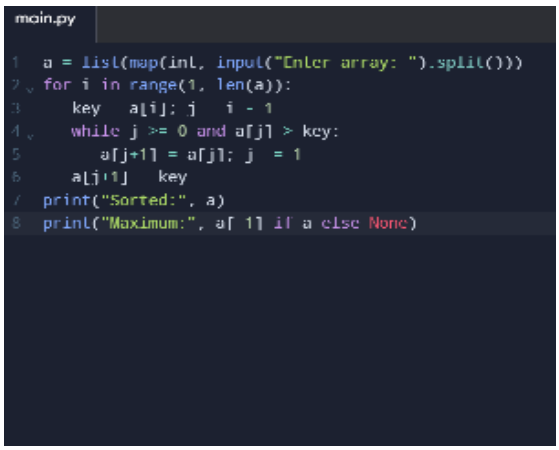
DAA Python Solutions Page 4

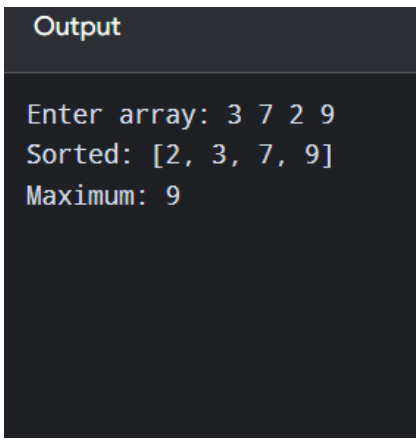
2. Sort the list in ascending order.

3. Return the last element.

Input: 3 7 2 9

Output: [2, 3, 7, 9]





Time Complexity: O(n²)

Space Complexity: O(1)

Result

The maximum is returned correctly, including empty and single-item lists.

Q7: Create a List of Unique Elements

Aim

To remove repeated values while preserving the first occurrence order.

Algorithm

1. Create an empty set and result list.
2. Visit each input value.
3. If it has not appeared before, add it to the set and result.
4. Return the result list.

Input: 3 7 3 5 2 5

Output: [3, 7, 5, 2]

```
a   list(map(int, input("Enter array: ").split()))
u = []
for x in a:
    if x not in u:
        u.append(x)
print("Unique list:", u)
```

```
Output

Enter array: 3 7 3 5 2 5
Unique list: [3, 7, 5, 2]
```

Time Complexity: O(n²)

Space Complexity: O(n)

Result

Only first occurrences are kept, in original order.

Q8: Bubble Sort

Aim

To arrange integer values in ascending order by repeatedly swapping adjacent out-of-order values.

Algorithm

1. Compare neighboring elements.
2. Swap if the left element is greater.

3. Repeat passes, reducing the unsorted range each time.

4. Stop when a pass makes no swaps.

Input: 5 2 9 1

Output: [1, 2, 5, 9]

```
a = list(map(int, input("Enter array: ").split()))
n = len(a)
for i in range(n):
    for j in range(0, n-i-1):
        if a[j] > a[j+1]:
            a[j], a[j+1] = a[j+1], a[j]
print("Sorted:", a)
```

```
Output

Enter array: 5 2 9 1
Sorted: [1, 2, 5, 9]
```

Time Complexity: O(n²)

Space Complexity: O(1)

Result

The list is sorted; early stopping improves already-sorted inputs.

Q9: Binary Search

Aim

To locate a key in a sorted array by repeatedly halving the search interval.

Algorithm

1. Set left and right boundaries.

2. Find the middle index.

3. If the middle value is the key, return its index.

4. Otherwise discard the half that cannot contain the key.

5. Return -1 if absent.

Input: 3 4 6 8 10 10

Output: 5

```
a = list(map(int, input("Enter sorted array: ").split()))
key = int(input("Enter key: "))
lo, hi = 0, len(a)-1
pos = -1
while lo <= hi:
    mid = (lo + hi) // 2
    if a[mid] == key:
        pos = mid; break
    if a[mid] < key: lo = mid + 1
    else: hi = mid - 1
print("Position:", pos + 1 if pos != -1 else -1)
```

```
Output

Enter sorted array: 3 4 6 8 10
Enter key: 10
Position: 5
```

Time Complexity: O(log n)

Space Complexity: O(1)

Result

Binary search works only on a sorted array. The example array was sorted before searching.

Q10: Merge Sort Without Built-in Sorting

Aim

To sort an array in O(n log n) time using divide-and-conquer.

Algorithm

1. Divide the list into two halves.
2. Recursively sort both halves.
3. Merge the sorted halves by comparing their first remaining elements.
4. Return the merged list.

Input: 5 3 1 4 2

Output: [1, 2, 3, 4, 5]

```
def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a)//2
    L, R = merge_sort(a[:m]), merge_sort(a[m:])
    out = []
    i = j = 0
    while i < len(L) and j < len(R):
        if L[i] < R[j]: out.append(L[i]); i += 1
        else: out.append(R[j]); j += 1
    return out + L[i:] + R[j:]

a = list(map(int, input("Enter array: ").split()))
print("Sorted:", merge_sort(a))
```

```
Output

Enter array: 5 3 1 4 2
Sorted: [1, 2, 3, 4, 5]
```

Time Complexity: O(n log n)

Space Complexity: O(n)

Result

Merge sort sorts without using a built-in sorting function.